Trabalho Laboratorial 0 — Parte 2

Ambiente, validação e ciclo de vida

Bernardo Pereira — 2022219048
João Lavos — 2021222529

1. Objetivo e configuração

A Parte 2 prolonga o trabalho da Parte 1 para um ambiente de desenvolvimento com Python, SDK, ROS 2, MoveIt 2 e Gazebo. Foram também implementados e validados os comportamentos de sessão segura e regresso à configuração home.

A validação foi realizada exclusivamente em simulação, não tendo sido usado o braço físico.

Elemento	Configuração utilizada
Nível intermédio	Windows, Python 3.12.10, ambiente virtual, SDK xArm e pacote local `lite6`.
Nível avançado	Via 1 — Docker, ROS 2 Jazzy, MoveIt 2 e Gazebo Harmonic.
Simulador de firmware	Contentor UFACTORY, perfil Lite 6 `6 9`, firmware v2.4.0 e IP `127.0.0.1`.
Identificação ROS	`ROS_DOMAIN_ID=0`, porque não foram atribuídos números aos grupos.

2. Validação do nível intermédio

Ponto	Resultado	Evidência
B.5.1	**PASSA**	`pytest -q`: 5 testes passaram.
B.5.2	**PASSA**	Ligação ao simulador e movimento start → home com `code=0`, tempo de 0.67 s e erro máximo de 0.000°.
B.5.3	**PASSA**	Não existem endereços `192.168` no código Python.
B.5.4	**PASSA**	O repositório possui commits e o README explica a recriação do ambiente e a execução dos testes.

A sessão segura reutiliza os comportamentos desenvolvidos na Parte 1: limpeza de avisos e erros, ativação dos servos, configuração do modo de posição, estado pronto, modo reduzido, limites de velocidade e fronteira cartesiana.

3. Testes avançados C.5

Teste	Verificação	Resultado	Observação
1	Comunicação ROS 2	**PASSA**	O listener recebeu mensagens `Hello World`.
2	Pacotes UFACTORY	**PASSA**	Foram encontrados 10 pacotes.
3	Modelo no RViz	**PASSA**	Lite 6 visível, `Global Status: Ok` e painel `MotionPlanning`.
4	Árvore de frames	**PASSA**	Foi gerado o PDF da cadeia `world` → `link_base` → `link_eef`.
5	Transformação tf2	**PASSA**	Foram registadas duas matrizes 4×4.
6	Gazebo independente	**PASSA**	As formas ficaram visíveis e o relógio de simulação avançou.
7	MoveIt no Gazebo	**PASSA**	`Plan & Execute` moveu o Lite 6 no Gazebo.
8	SDK contra tf2	**PASSA**	A cinemática direta devolveu `code=0`; diferença de 0.2000 mm na Pose A.

Não ocorreram falhas finais nos Testes 1–8, pelo que não existe uma primeira mensagem de erro a reportar.

4. Teste 5 — transformações e juntas

Pose A — configuração home

As juntas da Pose A foram:

$$q_A = [0, 0, 0, 0, 0, 0] \text{ rad.}$$

A matriz obtida com `tf2_echo`, em metros, foi:

$$T_A = \begin{bmatrix} 1 & 0 & 0 & 0.087 \\ 0 & -1 & 0 & 0.000 \\ 0 & 0 & -1 & 0.154 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Pose B

As juntas medidas pelo ROS 2 foram:

$$q_B = [0.858983,\ 1.378099,\ 1.051537,\ 0.000070,\ -0.326669,\ 0.858897] \text{ rad}$$

ou, em graus,

$$q_B = [49.216,\ 78.959,\ 60.249,\ 0.004,\ -18.717,\ 49.211]^\circ.$$

A matriz correspondente foi:

$$T_B = \begin{bmatrix} 1 & 0 & 0 & 0.134 \\ 0 & -1 & 0 & 0.156 \\ 0 & 0 & -1 & -0.024 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

No ROS 2, as translações são apresentadas em metros e os ângulos em radianos. O SDK utiliza milímetros e graus.

5. Testes 7 e 8

Teste 7 — Gazebo versus RViz

No Gazebo, o movimento foi executado pelo controlador simulado com dinâmica física, enquanto o RViz mostrou apenas o plano e o modelo cinemático, sem reproduzir os efeitos físicos da simulação.

Teste 8 — cinemática direta

Para a Pose A, o SDK devolveu:

$$\text{pose}_{SDK} = [87.0,\ 0.0,\ 154.2,\ 180.0,\ 0.0,\ 0.0].$$

A translação correspondente obtida com tf2 foi:

$$[0.087,\ 0.000,\ 0.154] \text{ m}.$$

A diferença entre as duas posições foi:

$$\Delta = [0.0,\ 0.0,\ -0.2] \text{ mm}$$

com norma:

$$\|\Delta\| = 0.2000 \text{ mm}.$$

Este valor confirma a consistência entre o SDK e o tf2 dentro da precisão das medições.

6. Reprodutibilidade D.4

A partir de uma shell nova, seguindo o README do grupo, foram repetidos:

- os testes Python do ambiente;
- a validação B.5 com movimento start → home;
- o Teste 1, com comunicação talker/listener;
- o Teste 3, com o modelo Lite 6 no RViz;
- o Teste 8, com comparação entre SDK e tf2.

Todos os testes foram reproduzidos com sucesso. O B.5 terminou com erro máximo de 0.000°, o listener recebeu `Hello World`, o RViz abriu com `Global Status: Ok` e o Teste 8 repetiu a diferença de 0.2000 mm.

As principais evidências são:

- PDF da árvore de frames em `shared/frames_2026-10-26_16.17.41.pdf`;
- CSV do ciclo de vida em `logs/lifecycle_sim_20261026_170724.csv`;
- juntas e matriz da Pose B em `docs/evidencias/`.

7. Exercício de ciclo de vida

7.1 Requisitos de `safe_session` e `home`

ID	Tipo	Requisito, métrica e limiar	Teste
R1	Funcional	`safe_session` deve ligar-se e deixar o controlador em modo 0, estado diferente de 4 e sem erros.	T-F1
R2	Funcional	A ligação deve ser fechada ao sair da sessão, incluindo uma exceção dentro do bloco.	T-F2
R3	Funcional	`home` deve terminar com código 0 e erro máximo das seis juntas $\leq 0.5°$.	T-F3
P1	Desempenho	Tempo de abertura: limiar < 5 s, alvo < 2 s e 10 repetições.	T-P1
P2	Desempenho	Tempo do regresso a home: limiar < 15 s, alvo < 8 s e 5 repetições.	T-P2
I1	Interface	Parâmetros provenientes de `robot.yaml`, sem IP fixo no Python e com unidades do SDK documentadas.	T-I1
S1	Segurança	Uma exceção durante o movimento deve provocar estado 4, deriva $< 0.05°$ em 1 s e alvo não atingido.	T-S1
S2	Segurança	Velocidade acima de `max_joint_speed` deve ser recusada com `ValueError` e sem movimento.	T-S2

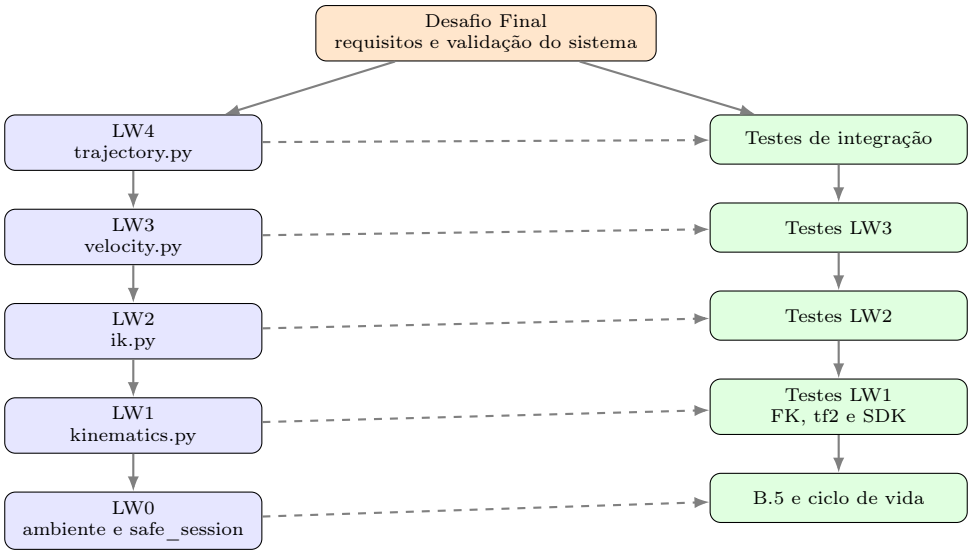
7.2 Resultados

Teste	Resultado	Valor medido
T-F1	**PASSA**	connected=True, mode=0, state=2, error=0.
T-F2	**PASSA**	connected=False após a exceção induzida.
T-F3	**PASSA**	code=0; erro máximo de 0.000°.
T-P1	**PASSA**	Pior 1.09 s; média 1.08 s; $n = 10$.
T-P2	**PASSA**	Pior 0.67 s; média 0.67 s; 5/5 homes válidos.
T-I1	**PASSA**	Teste automático sem IP fixo: 1 teste passou.
T-S2	**PASSA**	Velocidade recusada; movimento de 0.000°.
T-S1	**PASSA**	Estado 4 → 4; progresso 0.8°; deriva 0.000°; distância ao alvo 9.1°.

7.3 Matriz de rastreabilidade

ID	Módulo ou função	Teste	Script	Resultado
R1	`session.py::safe_session`	T-F1	`lifecycle_tests.py`	**PASSA**
R2	`safe_session`, bloco `finally`	T-F2	`lifecycle_tests.py`	**PASSA**
R3	`session.py::go_home`	T-F3	`lifecycle_tests.py`	**PASSA**
P1	`safe_session`	T-P1	`lifecycle_tests.py`	**PASSA**, 1.09 s
P2	`go_home`	T-P2	`lifecycle_tests.py`	**PASSA**, 0.67 s
I1	`config.py::load_config`, `robot.yaml`	T-I1	`test_config.py`	**PASSA**
S1	`safe_session`, `set_state(4)`	T-S1	`lifecycle_tests.py`	**PASSA**
S2	`session.py::_check_speed_cap`	T-S2	`lifecycle_tests.py`	**PASSA**

7.4 Modelo em V



8. Conclusão

O nível intermédio e a Via 1 do nível avançado foram validados com sucesso. Os oito testes avançados, os oito testes do ciclo de vida e a verificação de reprodutibilidade passaram no simulador utilizado.

Cada requisito de `safe_session` e `home` ficou associado a um módulo, a um teste e a um resultado na matriz de rastreabilidade.